

Liskov Substitution Principle

The L in SOLID, explained with a payment processing example

If code works with a parent type, it must keep working when you hand it a child type. No surprises.

The problem

```
class PaymentProcessor {
    void pay(Money amount) { }
}

class CreditCardPayment extends PaymentProcessor {
    private boolean otpVerified = false;

    void verifyOtp(String otp) { otpVerified = true; }

    void pay(Money amount) {
        if (!otpVerified) throw new IllegalStateException("OTP not verified");
        gateway.charge(amount);
    }
}
```

The caller was written against the parent:

```
void checkout(PaymentProcessor p, Money total) {
    p.pay(total);
}
```

UPI works. Cash on delivery works. Credit card crashes.

The parent promised "call `pay` and you're done." The child says "call `pay` only after I've verified an OTP." It demands more from the caller than the parent did, so it cannot substitute for the parent.

The usual patch makes it worse:

```
if (p instanceof CreditCardPayment) {
    ((CreditCardPayment) p).verifyOtp(askUser());
}
p.pay(total);
```

Once `checkout` has to ask what type it is holding, polymorphism has stopped doing its job. Every new payment method means editing this method again.

The fix

Keep the OTP inside the class that needs it.

```
interface PaymentProcessor {
    void pay(Money amount);
}

class CreditCardPayment implements PaymentProcessor {
    public void pay(Money amount) {
        String otp = otpService.requestAndCollect();
        gateway.charge(amount, otp);
    }
}

class UpiPayment implements PaymentProcessor {
    public void pay(Money amount) {
        upi.charge(amount);
    }
}
```

Caller, unchanged:

```
void checkout(PaymentProcessor p, Money total) {
    p.pay(total);
}
```

Both work. No `instanceof`. Add a fourth payment method and `checkout` does not change.

What actually changed

The OTP requirement never disappeared. It moved from the caller's responsibility to the card class's own responsibility, where it belongs.

The rule: a subclass may do more than its parent. It may never ask more of the caller than its parent did.

The tell in review: a subclass that throws where the parent returned, or a caller that needs `instanceof` to use it safely.